

BACKUP RECOVERY

STRUCTURES USED FOR BACKUP :

1) Datafiles :

It stores actual data from which retrieval is done and DML operations are performed.

2) Logfiles :

It stores history of DML as well as DDL operations in low level format. All the entries are identified using SCN .If the object is created using no login option then no log entry is generated for that object. The log entries are generated using server process.

3) Control files :

It stores physical structure of the database i.e. actual path of data files and log files. The path of the control file is stored in the parameter file.

4) Table Space :

It is a group of data files which are having same logical data.
Ex. PAYROLL

5) Rollback Segment & Undo Segment :

It is having undo information of transaction.

6) Parameter File :

It is having parameters for specifying archiving, multiplex of log files and control files.

CONCEPT OF RECOVERY :

Oracle decides whether recovery is required using information from control file and data headers and log file headers are used as follows:

log 1	log 2	log 3
FIRST SCN =1 Last SCN=91		
FIRST SCN=92 Last SCN=139		
FIRST SCN=140 Last SCN=251		

datafile1	datafile2	datafile3
Checkpoint SCN=275		
Checkpoint SCN=275		
Checkpoint SCN=197		

If Checkpoint SCN of any datafile is less than maximum of any online data file then that datafile is supposed to have old consistent image.

For ex. Datafile 1

For such datafile recovery has to be applied. And which log has to be applied is decided according to last SCN. In this case for recovery of datafile1 entries having no. 166 onwards has to be applied which is present in log 3. So all the log files from log 3 are used.

RECOVERY DETECTION USING CONTROL FILES :

Control file is having two fields START SCN and STOP SCN .

When database is shutdown both are having same value i.e last checkpoint SCN . For ex. Start SCN =131

Stop SCN =131

When the database is open then start SCN will have certain value while Stop SCN is having value INFINITY. When there is instance failure because of certain reason like power failure then stop SCN will have INFINITE value. It indicates that recovery operation has to be done for all data files and all log entries are applied so that data files are having changes up to last committed SCN.

Committed transactions are redone and uncommitted transactions are undone.

Example :

Suppose start SCN = 312 and stop SCN =INFINITY means database is open. Let log entries are written upto 351. During recovery oracle try to apply entries from 312 to 351. 312 indicates that minimum SCN which is used for recovery. If datafile SCN is having value 327 then entries from 328 to 351 are applied.

CONCEPT OF CHECKPOINT :

Checkpoint means recording state of the database, which is present at certain point of time. Suppose operations are done on two data items A and B as follows

	State	A	B
101 T1 start			
102 T1, A, 100, 90	1	100	100
103 T1, B, 100, 110	2	90	100
104 T1, COMMIT	3	90	110

At any point of time database can have as a state 1, 2 or 3. Since the changes are done first in memory and then in data file, image of the data file constructed from memory and data file is always consistent. But because of independent buffer management image recorded in data file may not be consistent.

For ex: A=100 & B=110

This case arise when buffer B is written first than A. The value A=100 & B = 110 is not pertaining to consistent image because it was never in existent at certain point of time. Thus image in the data file can be inconsistent. This also indicates that copying the data file when the system is online does not make consistent backup.

Checkpoint ensures that some consistent image is recorded in data file. For ex. The checkpoint starts after SCN=102, it means that there is a guarantee

that A = 90 and B = 100. If B is already recorded then it may have value 110.

During checkpoint all dirty buffers up to that SCN are written in data file and that data file header is updated for that SCN. Suppose checkpoint starts after 102 then buffer A is written on disk and data file header indicates checkpoint SCN as 102. It means that recovery for data file is done from SCN 103. Checkpoint is time-consuming process if there are many dirty buffers. Dirty buffers are written by DBWR and headers are updated using CKPT process. Checkpoint can be implicit or explicit. If the user is not aware of checkpoint then it is implicit checkpoint. For ex: checkpoint during log file switch.

In the case of explicit checkpoint user is aware of checkpoint. For ex: when the user issues the command `Alter_System_Checkpoint` or `checkpoint` which is done is begin backup.

Checkpointing is done for all database or it can be done for individual tablespace. When there is a database checkpoint then entry is added in the log file regarding completion of checkpoint. The checkpoint is also categorized as fast checkpoint and slow checkpoint. In the case of fast checkpoint all dirty buffers are written on disk and headers are updated, so in that period oracle is not doing any job. This is done generally for explicit checkpoint.

In the case of slow checkpoint oracle adds only `<start checkpoint>` entry in log file and buffers are written to complete this checkpoint. So the buffers are stored in ascending order of SCN and they are written in that sequence. During this buffer writing other operations are also allowed.

When required buffers are written then datafile headers are updated and `<end checkpoint>` record is added in log file. If checkpoint is not completed then previous checkpoint is used for reference. Whenever there is a log switching slow checkpoint is used. During log file switch slow checkpoint starts. Suppose it is not completed when log file is reused then event log switch stuck is generated and slow checkpoint is converted into fast checkpoint. This is necessary in Noarchive log mode.

Que: Why checkpoint is essential at the time of log switch in no archive log mode?

In the noarchive log mode only log files, which are used, are having information if there are two log files then redo information of two files is only available.

Suppose there is a log switch from log1 to log2 with last SCN = 101 for log1. After that writing starts in log2. When log2 is over then there is a switch to log1 and it is overwritten.

Recovery requires some base backup and all the log entries after that when log1 is rewritten, it should be ensured that all the changes upto 101 are written in database. So that for recovery log entries after 101 are required which is available in 102. So checkpoint starts after log switch and it is slow

checkpoint. If the checkpoint is not completed till log2 is over then to complete a checkpoint the previous checkpoint is converted to fast checkpoint.

CHECKPOINT OF TABLESPACE :

Tablespace checkpoint is done when command 'Alter tablespce < tablespace name > begin backup' is given.

In response to this command dirty buffer of that tablespace are written to datafile and headers of the datafiles of that tablespace are updated. This command is given at Sql prompt. When the command is completed then datafiles of that tablespace are copied to backup location using operating system command. In this mode the operation on that table space are also permitted. This method is called as **online backup** or **hot backup**. In this case oracle block may be written by DBWR and it may be read concurrently by OS command. So in some case inconsistent image of the block is recorded.

Suppose one oracle block consistent of 4 operating system blocks. And if after copying first two OS blocks DBWR is written complete oracle block. Then block no 3 and block no 4 will have new data. So the copy image of block may be

1	
2	
3	
4	
101	
Old	
Old	
New	
New	
102	

Oracle block

This is sort using block versioning. Oracle records change version of the block in the header portion and trailer portion. Using version numbers the block having inconsistent images are deleted. To get the consistent image oracle stores entire block into redo log file after begin backup command. In this period the rate of redo generation is very high. After backing up the data files user can indicate using <alter DoubleSpace> <DoubleSpace name> using end backup command. After this command the redo generation is according to normal rate. The interval between begin backup and end backup should be minimum.

INITIALIZATION PARAMETERS WHICH AFFECTS CHECKPOINT :

1. LOG_CHECKPOINT_INTERVAL=1024379
It indicates that after this many redo log generation checkpoint should begin.
2. LOG_CHECKPOINT_TIMEOUT=1900
It indicates time intervals in seconds to start intervals.
3. LOG_CHECKPOINT_TOALERTS
If this is true then checkpoint information is added to alertfile.
4. FAST_START_MTTR_TARGET=900
It is given in seconds and it indicates that recovery time during instance recovery is approximately to these seconds (time for recovery).
5. FAST_START_IOTARGET
It is deprecated parameter. It limits the recovery time in terms of no. of redo generation. It limits the recovery time by controlling population of dirty buffers. If these parameters are set then oracle maintains queue of the dirty buffers in increasing order of their SCN and these buffers are written according to their SCN values. This will disturb the LRU policy of buffer management.

ROLL OF UNDO SEGMENT IN RECOVERY:

When the checkpoint is done then there can be some active transaction. In principle for

recovery only the log after the checkpoint is considered. But if some transaction is not completed after checkpoint then it will require entries before the checkpoint for UNDO information. For eg. If the log is generated in following sequence

<T1,Start>
<T1,A,100,90>
<T1,B,100,110>
<T2,Start>
<Checkpoint>
<T2,C,100,110>
<T2,COMMIT>

In this case T1 is active transaction & T2 is committed transaction at the time of crash. Oracle maintains transaction table for active transactions.

At the point of crash, T2 is committed and T1 is uncommitted transaction. Transaction table always points to undo information in undo segments. In case of no archive log mode, information from undo segment is used to nullify the effect of the transaction because amount of redo is limited in no archive log mode.

In archive log mode transaction table along with redo information is dumped into log file along with checkpoint. So that during recovery the transaction table can be constructed & it can be used for undoing effect of transaction.

In no archive log mode if there is media failure in undo segment the recovery is not possible. While in archive log mode even if there is media failure, recovery is possible.

HOW RECOVERY IS DONE

During recovery, first transaction table is constructed from latest valid checkpoint. Either it is constructed from undo tablespace or from log file.

Suppose transaction table is having active transaction like T1, T2, T3, T4 then redo log is scan & changes are applied. If transaction is committed then it is removed from transaction table.

If new transaction started after checkpoint then it is added in transaction table. This phase is called as rolling forward. At the end of rolling forward phase, you will have transactions which are not committed. So these transactions are rolled back.

During recovery Undo segment is required for Undo transaction.

HOW YOU WILL SECURE YOUR SYSTEM

To secure the system following actions are done:

1. Running the system in archive log mode.
2. Multiplex log files.
3. Multiplex archive files.
4. Multiplex control files.
5. Copy the control files to back up locations after change in the physical structure of database after dropping the data files.
6. Provide consistent backup for each tablespace .
7. Provide full database backup if it is possible.

1. **Putting the database in archive log mode** : After Oracle is installed and basic tablespaces are created & data is loaded .Shutdown database normally.

Modify parameters as follows :

- 1) LOG_ARCHIVE_START =TRUE :

It will start archive process for automatic archiving.

- 2) Copy the existing control files to multiple locations preferably on separate disk.
This locations are called as mount point and specify the location in parameter file for parameter CONTROL_FILES.

- 3) You can specify archive location using parameter

LOG_ARCHIVE_DEST=D:\ backuparch

ALTER DATABASE ARCHIVELOG

If you want to provide multiple locations

LOG_ARCHIVE_DEST_1 = (LOCATION = D :\ backuparch)

LOG_ARCHIVE_DEST_2= (LOCATION = D :\ backuparch)

LOG_ARCHIVE_DEST_3 = (LOCATION = D :\ backuparch)

|
|
|

LOG_ARCHIVE_DEST_10 = (LOCATION = D:\ backuparch)

If you are providing destination as log archive dest another location you give
LOG_ARCHIVE_DUPLEX_DEST :-

If multiple locations are given because of media problem, archiving may not be done at all places.

LOG_ARCHIVE_DEST_STATE_2 = DEFER :-

Another value is enable.

2. Multiplexing of log file is done by adding log file members in log groups .
Alter database add logfile member

Alter database

Add logfile member 'c:\log \log 01.dbf ' to GROUP 1

Groups can be retrieved from v\$log & existing members can be retrieved from v\$logfile . You can add the groups using commands .

Alter database

Add logfile Group 4 ('c:\log \log 04.dbf ' , 'd:\log \log 04.dbf ') size 10M

Commands can be given after monitoring database.

HOW TO TAKE FULL DATABASE BACKUP

1. Full database backup means copying all datafiles ,control files ,log files and parameter files to backup locations . These can be used as base backup for next recovery step.
2. Identify data files using v\$datafiles.
3. Identify log files using v\$logfiles.
4. Identify control files using v\$paracontrolfiles.
5. Full db backup is taken only after shutting db normally.

PRECAUTIONS WHEN SYSTEM IS RUNNING

1. When some physical changes are done in db like adding tablespace or adding datafiles then after making the change backup that object.
2. If you cannot shutdown the system then you take the hot backup of tablespaces.
ALTER TABLESPACE PAYROLL BEGIN BACKUP;
--COPY ALL DATAFILES OF PAYROLL TABLESPACE
ALTER TABLESPACE PAYROLL END BACKUP;
3. Backup the control file using command
'Alter database backup control file to <locations>
4. For any future recovery this control file has to be used.
5. Old control file could not be used for recovery.

RECOVERY COMMAND

1. Recover Database :
It recovers complete database.
2. Alter Database :
Recover tablespace tablespacename
3. Alter Database :
Recover datafile datafilename

Recover database apply all log files :

When you want to apply log upto particular SCN or log sequence no. then you can use the command :

1. Alter database recover automatic until time '2004-10-27 :14:20:00'
2. Recover database until cancel.
3. Alter database recover automatic until cancel.

Automatic means it takes the log file names from log archive destination and log archive format.

Whenever you are recovering database , it should be mounted while database & tablespaces can be recovered after opening datafiles.

If database is running in archive log mode then opening database requires all datafiles needs recovery .

1. If db is running in archive log mode then make the file offline using cmd :
Alter database datafile <datafile path > offline
After opening database you can recover datafile & make it online.
Datafile related with system tablespace should be online for opening database.
2. If db is running in no archive mode then you cannot make datafile offline . Instead of that you have to use the option offline drop. In this case data will be lost.

If certain datafile is offline for certain period then it is recorded for control file as offline scn.

So during recovery this scn are not considered for datafile.

PROBLEMS IN DATABASE

There are two major problems

1 . **Instance failure** like database is running and certainly there is power off. This can be recovered in archive log mode as well as noarchive log mode .

2. **Media Failure** , It includes loss of datafile ,control file ,log file ,etc.

In archive log mode this can be recovered while in noarchive log mode control file can be recovered but datafile ,logfile ,etc cannot be recovered . The only option is restore the backup & import from latest export.

D1 D2 D3 D4 D5 D6 D7 D8

BCK

L10 L11 ..L30 L31

MOUNT DATABASE -STARTUP MOUNT

RESTORE OLD BACKUP - OS -RESTORING ALL DATAFILES

APPLY LOG FILES UPTO L30- RECOVER DATABASE UNTIL CANCEL

ALTER DATABASE OPEN RESETLOGS

FULL DATABASE BACKUP

DELETE ALL EXISTING BACKUPS AND ARCHIVE LOG FILES

1) LOSS OF DATAFILE

1. Offline recovery :

- a) Mount the database
- b) Restore datafile from backup location.
- c) Recover datafile : this cmd prompts user to supply log file names which are to be applied .

2. online recovery :

- a. Restore datafile from bacup.
- b. Mount database
- c. Make datafile offline.
- d. Open database .
- e. Recover datafile.
- f. Make datafile online.

This is how datafiles are recovered for any tablespace including system tablespace and undo tablespace.

2. LOSS OF DATAFILE & DATAFILE CANNOT BE RESTORED THAT ORIGINAL POSITION

- a. Mount the database.
- b. Restore the datafile at new location .
- c. Rename file i.e changing the path of the file using cmd :
Alter database rename file <old path> to <new path>
- d. Restore file and then open the database.

Media Recovery can be of two types :-

- 1) **Complete media recovery**
- 2) **Incomplete media recovery**

In normal case , complete media recovery is used. Incomplete media recovery is used When there is a loss of current log file or you want to recover some dropped table. In the incomplete media recovery some recent data may be lost. In the case of incomplete media recovery ,after backup is restored,some recent log files are not applied. So you will get the state of database which is in existence at certain previous time.

For example, if you drop table at 3'O clock and at 7'Oclock you realised that table

Is required so you want to restore the state of the database prior to 3'oclock .so in this case some previous backup is restored say of yesterday and logs prior to 3'oclock is applied. This can be done using command-

> Recover database

Until <time>

In the case of incomplete media recovery existing archive information and log information becomes useless. So oracle has to create new log files. So you have to open the database in reset log mode. In the resetlog mode,log files are newly created. If you can't create the members at original location then after mounting the database ,you have to rename the data file. To avoid the conflict of old archive files,you have to delete existing archive files. After opening the resetlog ,the old backup become useless because you don't have old log file and archive file for that. After opening resetlog,shutdown the database normally and take the backup which acts as base backup for next recovery. So the steps for incomplete media recovery are as follows :-

- 1) Restore full base backup i.e. restoring all data file.
- 2) Use command

Recover database

Until cancel

Or

Recover database

- Until time
- 3) Alter database
Open resetlog
 - 4) Shutdown database normally
 - 5) Take the backup of database for next recovery problem.
 - 6) Delete all existing archive files.

Loss Of Control File :

If all control files are lost then control file can be created using **create control file** command. In this command you have to mention structure of database. Generally this command is generated during 'Alter database backup control file to trace' option. This is done when system is running normally.

SQL> Alter database backup controlfile to trace;
The command dumps output in trace file.

Loss Of Archive File :

If archive files are accidentally deleted then take the backup of the database for recovery which will act as base backup.

Loss Of Temporary Tablespace :

If temporary tablespace is lost then you can switch over to another temporary tablespace. So as a precaution you should always have two temporary tablespace.

In the case of loss of undo tablespace ,same action is needed. But in this case database shutdown is required and recovery is needed.

Instance Crash When Tablespace In Backup Mode :

- 1) start the instance and mount it.
- 2) Give command - Alter tablespace<tablespace-name>
End backup
- 3) Open the database

RMAN (Recovery Manager) :

Advantages of RMAN :

- 1) Recovery and backup can be managed from remote location.
- 2) Backup maintainance is easy. Only that part is backupd which is used .
For eg . if there is 100 MB file but only 40 MB file is used,then data of only 40 MB is copied.
- 3) Incremental backup can be done. So backup will take less time.
- 4) Backup files can be made according to specification i.e. even if data file is 500 MB ,it can be cut into 50 MB pieces.

Architecture of RMAN :

Recovery manager stores the information of the database which is required for backup and recovery. This is called as repository. In the set of database servers one database server is exhaustively used for RMAN and it stores information about all other servers. For backup and recovery operation, administrator has to connect to RMAN server and carry out the operations.

Initial Setup Using RMAN :

- 1) Add connect string in tns -> admin for RMAN server.
- 2) Check the connection using command -tnsping.

RMAN server is also known as Catalog Server and server for which you want to store information is known as Target Server. In this case Nldap is target server and orcl83 is the catalog server.

- 3) c:\backup\sql\RMAN (to start RMAN)
- 4) connect to target server.
- 5) Connect to the catalog server
- 6) Register database (it stores the repository information in catalog)
- 7) Backing up tablespace users using RMAN .

Write the following script in file let backup9_1.txt

```
RUN
{
    Allocate channel disk\device type disk format
    'c:\backup\bck\ts%u'
    Maxpiecesize=200M;
    Backup tablespace users maxsetsize =400 M channel "DISK1";
}
```

channel indicates physical specification of output backup file.

Format indicates name and location of file.

If the character precedes with %, it is having special meaning.

U -> some unique identifier

T,S -> prefix

MAXSETSIZE -> maximum size of file.

You can view the file at that location.

Generally this file is having less size than actual size because only those blocks are copied which are having data.

Backup indicates what you want to backup. You can backup database, tablespace, datafile and you have to indicate the channel in backup. You can give command

list backup

to find out information about the backup. Each backup set is identified using backupset key (BS key).

Recovery Using RMAN :

- Shutdown the system, rename data file user01.dbf (say) so that it will not be available.

RMAN> restore tablespace users

When you restore, that file is backup.

```
RMAN>SQL 'alter database open';
RMAN>SQL 'alter database users online';
```

Scripts :

Backup1.script

```
RUN
{
  Allocate channel disk\device type disk format 'c:\backup\bck\RMAN
%u'
  Backup database ;
}
```

It makes one backup set of all data files and control files and one output file . one backup set is having multiple data files and using RMAN you can make multiple backup set using single script.

One set can be split up into multiple backup pieces. For each backup piece there is one backup file. For eg. If there are 10 data files d1 ..d10 ,then you can have flexibility in defining backup set and backup pieces. Suppose you want 4 backup sets and for each backup set you want 2 backup pieces then you can have combination like

backup set1=d1,d2,d3

backup set2= d4,d5,d6

backup set3=d7,d8

backup set4=d9,d10

you can have 8 backup files. And if you don't specify then one backup set and one backup piece.one datafile can be member of more than one piece.

Backup2.script

```
RUN
{
  Allocate channel disk\device type disk format 'c:\backup\bck\RMAN%u'
  Backup database filesper set=2;
}
```

It will make backup sets such as in 1 set,there will be maximum 2 files and each set is having app. Same size and for each backup set ,there is only one piece .

Backup3.script

```
RUN
{
  Allocate channel disk\device type disk format 'c:\backup\bck\RMAN%u'
  Backup database filesper set=2 maxsetsize =400 M;
```

```
}
```

it will make the backup set such that file per set is 2 and maxsetsize =specified size.setsize is sum of size of data file .

Backup4.script

```
RUN
{
  Allocate channel disk\device type disk format 'c:\backup\bck\RMAN
%u'
  Backup database;
  maxpiecesize =50 M;
}
```

it will make one backup set and it is divided into 50 MB backup pieces.

Backup5.script

```
RUN
{
  Allocate channel disk\device type disk format 'c:\backup\bck\RMAN%u'
  Backup database;
  maxsetsize =400 M;
  maxpiecesize =50 M;
}
```

it will make the backup set such that maxsetsize =400 M and each setsize is divides into 50M pieces.

Backup6.script

```
RUN
{
  Allocate channel disk\device type disk format 'c:\backup\bck\TS%u'
  Backup tablespace users
  maxsetsize =400 M;
  maxpiecesize =200 M;
}
```

it is similar to previous but it backup the tablespace.

Backup7.script

```

RUN
{
  Allocate channel disk\device type disk format 'c:\backup\bck\TS%u'
  Backup incremental level 1 cumulative tablespace users
  maxsetsize =400 M;
  maxpiecesize =200 M;
}

```

it is cumulative backup and it starts with level1 so if there are 200 used blocks then it will make backup of 200 blocks. If after level1 backup 50 blocks are changed then in level2 backup 50 blocks are copied. After level 2 backup if 45 blocks are changed next level2 backup will copy 45 blocks.

When backup is restored ,it is reconstructed from 200 + 50+45 blocks. Cumulative backup takes less time.

Backup9.script

```

RUN
{
  Allocate channel disk\device type disk format 'c:\backup\bck1\TS%u'
  maxpiecesize =200 M;
  Allocate channel disk\device type disk format 'c:\backup\bck2\TS%u'
  maxpiecesize =200 M;
  Backup tablespace users maxsetsize =400M channel "disk1" ;
  Backup tablespace tools maxsetsize =400M channel "disk2" ;
}

```

channels are used so that you can have different specifications for different backup sets. The users tablespace is having backup pieces in bck1 directory and tools having backup pieces in bck2 directory.

Backuparch.txt

```

RUN
{
  Allocate channel disk\device type disk format 'c:\backup\bck\TS%u'
  Backup archivelog all;
  maxpiecesize =200 M;
}

```

it is making the backup set of all archive files instead of data files. Archive files are collected

from log_archive_dest.

Backup10.txt

```
RUN
{
  Allocate channel disk\device type disk format 'c:\backup\bck\TS%u'
  Backup validate tablespace tools;
  maxpiecesize =200 M;
}
```

it makes the backup and validate the backup pieces by scanning the backup files.

Backup12.txt

```
RUN
{
  Allocate channel disk\device type disk format 'c:\backup\bck\TS%u'
  Backup copies to datafile 7 format 'c:\backup\bck1\TS%u',
  'c:\backup\bck2\TS%u';
  maxpiecesize =200 M;
}
```

It make the two copies of datafile 7 .one copy is stored in BCK directory and other in BCK2 directory.

Backup14.txt

```
RUN
{
  Allocate channel disk\device type disk format 'c:\backup\bck\control
  %u'
  Backup current controlfile
  maxpiecesize =200 M;
}
```

It make the current control file backup.

DelBackup 2.txt

RMAN> Delete backup;

It will delete all the backupset interactively.

DelBackup 1.txt

RMAN> Delete backupset 34;

It will delete backupset no. 34.

Det_block5.txt :

Select file #1,incremental_level,completion time,blocks,datafile_block5 from
v\$backup_datafile order by file#,completion_time;

r_datafile.txt:

If you don't have backup of certain datafile but you are having all log files then you can recover data using following command

```
RUN
{
  Allocate channel disk\device type disk format
  'F:\backup\RMANBCK\RMAN%u'
  SQL "alter database create datafile
  'F:\oracle9i_db\oradata\psd12\users02.dbf';
  Recover database;
}
```

Standby Backup :

In standby configuration, there are two database servers. First server is known as primary server and second is standby server. All the operations are done on primary server and they are transferred to standby server using log file. To start the standby arrangement ,datafiles of primary server are copied to standby server.

On the primary server,you have to make control file using command -----

```
SQL>Alter database create
Standby controlfile
As '/oracledbs/stbycf.f';
SQL>startup nomount pfile=D:\dbbackup\init.ora
SQL>Alter database mount standby database;
SQL> Alter database recover managed standby database;
```

Control file is copied to standby server and standby server is started with the parameterfile which indicates this control file in parameter control files.

You can start the standby database using command-----

```
SQL>startup nomount pfile=D:\dbbackup\init.ora
SQL>Alter database mount standby database;
```

The parameter control file should indicate standby control file.

#File Configuration :

```
control_files=(“d:\dbbackup\standby.ctl”);
```

since now standby database is mounted using standby control file. Standby control file indicate physical structure of primary database. Structure of the primary database may not be according to location of datafiles in standby datafiles.

For eg.. in primary database,user file users.dbf is having path C:\oradata\users.dbf . in the standby it is copied to D:\standby\users.dbf so in the standby database ,you can make the changes in the control file using command-----

```
SQL> Alter database rename file ‘C:\oradata\users.dbf’ to ‘D:\standby\users.dbf’
```

For each such database ,renaming is done. You can use parameter db_file_name_convert at log_file_name_convert in parameter file of standby database.

These are used for substituting source string by target string in file path.

#Archive Parameters :

```
db_file_name_convert=(‘C:\oradata’, ‘D:\standby’)  
log_file_name_convert=( C:\oradata’, ‘D:\standby’);
```

In the standby server oracle reads path ‘C:\oradata\users.dbf’ using pair given in db_file_name_convert substitute using ‘D:\standby’ and generate new path a ‘D:\standby\users.dbf’ since the databasename of both servers are different in parameter file of standby database , database_name is made equal to primary database.

After making the changes in parameters file ,standby database is started. If you have to make archive for standby database then you have to set database as standby_archive_dest. On primary server,you have to make only one change i.e. adding another archive destination.

```
Log_archive_dest_2=’service=psd12’ arch
```

Where psd12 is connect string for standby database. Connect string is entered in tnsnames.ora in primary database.

Dest_2 indicate when archive is generated ,it is sent to the standby database. After making the changes in parameter file ,first standby server is started in the recover managed mode using following commands-----

```
SQL>startup nomount pfile =D:\dbbackup\init.ora;
```

```
SQL>Alter database mount standby database;
```

```
SQL> Alter database recover managed standby database;
```

When archive is generated on primary side,it is sent to the standby server.

How Standby Works ?

Whenever the log is generated on the primary side, it is supplied to standby database. On the standby database, it is applied to base backup so that data in the standby is updated. So standby is one log behind primary database. If there is some problem in primary database then after mounting database,

You have to make log switch so that the current log file is sent to standby and state of both database

Servers remain same. If it is not possible, manually copy the log file to standby database and apply the log file using command---

>recover database until cancel;

After that database is open using command-

>Alter database open standby database;

After opening the standby database, connection between the primary and standby database is broken. If you want to again make standby database you have to start from copying the database.
